

Combinational Logic Design using VHDL

Example:

Provide VHDL implementation of the circuit shown in figure 2. Figure 1 shows the black box design of the circuit and figure 2 depicts the internal details. The three design styles to be followed are (i) Dataflow design style, (ii) Behavioral design style, and (iii) Structural design style

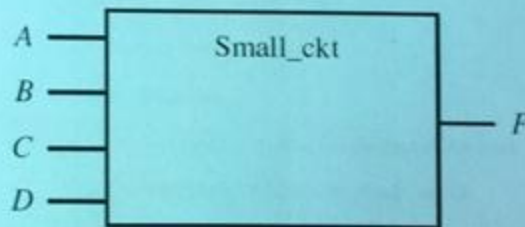


Figure 1: Black Box Design of Small_ckt

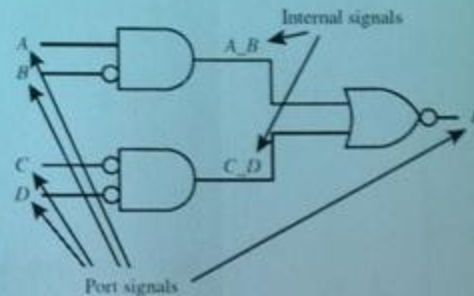


Figure 2: Internal Details of Small_ckt

The term *black box* specifies only the external signals for a design entity and not its contents. A black box is equivalent to a symbol in a schematic. Small_ckt (short for Small_circuit) is the name that is used to identify the design entity.

(a) Dataflow Design Style (using a Boolean Equation)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
  A, B, C, D : IN STD_LOGIC;
  F : OUT STD_LOGIC
);
END Small_ckt;

--This is a dataflow architecture declaration for Small_ckt

ARCHITECTURE dataflow OF Small_ckt IS
BEGIN
  F <= (A AND NOT B) NOR (NOT C AND NOT D);
END dataflow;
```

Figure 3: VHDL Code for Small_ckt in Dataflow Style (Boolean Equation)

- A Boolean expression is assigned to the signal (F) via the signal assignment symbol (<=)
- A semicolon is required to terminate the simple signal assignment statement
- VHDL has no order of precedence for logic operators with two signals; therefore, parentheses must be used to establish precedence
- NOT has the highest precedence of all the operators in VHDL and so does not need parentheses

Combinational Logic Design using VHDL

- The evaluation of a Boolean expression provides the selection value
- Parentheses determine the order of precedence for logic operators
- Only one signal assignment symbol (<=) is used for a selected signal assignment
- A single bit is represented as '1' or '0', i.e. using single quotation marks
- The selected signal assignment examines the value of the expression, and executes only the signal assignment that matches the WHEN value, and all the other signal assignments are skipped
- WHEN OTHERS is required to terminate a selected signal assignment statement
- WHEN OTHERS is used to ensure that all possible select values for the Boolean expression are included in the choice list after WHEN
- Commas are required in a selected signal assignment

(d) Behavioral Design Style (using a Process Statement and Boolean Equations)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
    A, B, C, D : IN STD_LOGIC;
    F : OUT STD_LOGIC
);
END Small_ckt;

--This is a behavioral architecture declaration for Small_ckt

ARCHITECTURE behavioral OF Small_ckt IS

    SIGNAL A_B, C_D: STD_LOGIC;

BEGIN

    example: PROCESS (A, B, C, D, A_B, C_D)

    BEGIN
        A_B <= A AND NOT B;
        C_D <= NOT C AND NOT D;
        F <= A_B NOR C_D;

    END PROCESS example;

END behavioral;
```

Figure 6: VHDL Code for Small_ckt in Dataflow Style (Process and Boolean Equations)

- Internal signals (A_B and C_D) must be declared within the declarative part of the ARCHITECTURE, because it is illegal in VHDL to declare signals in a PROCESS
- The PROCESS statement contains a list of signals called the sensitivity list. A PROCESS statement executes when an event occurs on any one of the signals in the sensitivity list
- All inputs must be included in the sensitivity list, including the internal signals
- A PROCESS has a BEGIN and an END, and each statement in a PROCESS ends with a semicolon
- Each statement in a PROCESS is executed in the order that it is written in the PROCESS
- Boolean equations can be used inside a PROCESS, but conditional signal assignments and selected signal assignments cannot be used inside a PROCESS

Combinational Logic Design using VHDL

(b) Dataflow Design Style (using a Conditional Signal Assignment)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
  A, B, C, D : IN STD_LOGIC;
  F : OUT STD_LOGIC
);
END Small_ckt;

--This is a dataflow architecture declaration for Small_ckt

ARCHITECTURE dataflow OF Small_ckt IS
BEGIN
  F <= '1' WHEN ((A AND NOT B) NOR (NOT C AND NOT D)) = '1' ELSE
    '0';
END dataflow;
```

Figure 4: VHDL Code for Small_ckt in Dataflow Style (Conditional Signal Assignment)

- Only one signal assignment symbol (<=) is used for a conditional signal assignment
- A single bit is represented as '1' or '0' i.e. using single quotation marks
- A Boolean expression or signal may be compared to a bit using the "=" relational operator
- Parentheses must be used around a Boolean expression when it is compared to a bit using a relational operator
- Parentheses determine the order of precedence for logic operators
- A semicolon is required to terminate a conditional signal assignment
- The ELSE after the last WHEN is needed to avoid non-functional circuits

(c) Dataflow Design Style (using a Selected Signal Assignment)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
  A, B, C, D : IN STD_LOGIC;
  F : OUT STD_LOGIC
);
END Small_ckt;

--This is a dataflow architecture declaration for Small_ckt

ARCHITECTURE dataflow OF Small_ckt IS
BEGIN
  WITH (A AND NOT B) NOR (NOT C AND NOT D) SELECT
    F <= '1' WHEN '1',
    '0' WHEN '0',
    '0' WHEN OTHERS;
END dataflow;
```

Figure 5: VHDL Code for Small_ckt in Dataflow Style (Selected Signal Assignment)

Combinational Logic Design using VHDL

(e) Behavioral Design Style (using Process Statement & If-Then-Else Statement)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
    A, B, C, D : IN STD_LOGIC;
    F : OUT STD_LOGIC
);
END Small_ckt;

--This is a behavioral architecture declaration for Small_ckt

ARCHITECTURE behavioral OF Small_ckt IS
BEGIN
    PROCESS (A, B, C, D)
    BEGIN
        IF (A = '1' AND B = '0') NOR (C = '0' AND D = '0') THEN F <= '1';
        ELSE F <= '0';
        END IF;
    END PROCESS;
END behavioral;
```

Figure 7: VHDL Code for Small_ckt in Dataflow Style (Process and IF-THEN-ELSE Statement)

- A PROCESS statement is used within the BEGIN and END of the ARCHITECTURE
- A PROCESS statement may not be named, because naming a PROCESS is optional
- A PROCESS contains a list of signals called the sensitivity list
- A PROCESS executes when an event occurs, i.e. when a signal in the sensitivity list changes
- Semicolons are required after the simple signal assignment statements, and also after END IF
- An IF-THEN-ELSE statement must have the last ELSE statement to avoid non-functional circuits
- An IF-THEN-ELSE statement must be placed in a PROCESS between BEGIN and END PROCESS
- To handle multiple conditions, IF-THEN-ELSE statements can be nested
- An IF-THEN-ELSE statement is similar to the conditional signal assignment

(f) Behavioral Design Style (using Process Statement & Case Statement)

- A CASE statement must be placed in a PROCESS between BEGIN and END PROCESS
- The CASE statement examines the value of the signal that is listed after CASE, and only executes the output statement to the right of the symbol => that matches the WHEN value for the signal, and all other output statements are skipped
- WHEN OTHERS statement is used to ensure that all possible select values for the signal listed after CASE are included in the choice list after WHEN
- The keyword NULL means perform no action
- A semicolon is needed after each signal assignment, and also after the keyword NULL to terminate the case statement
- The CASE statement is similar to the selected signal assignment

Combinational Logic Design using VHDL

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

--This is the entity declaration for Small_ckt

ENTITY Small_ckt IS PORT (
  A, B, C, D : IN STD_LOGIC;
  F : OUT STD_LOGIC
);
END Small_ckt;

--This is a behavioral architecture declaration for Small_ckt

ARCHITECTURE behavioral OF Small_ckt IS

BEGIN

PROCESS (A, B, C, D)

BEGIN

CASE (A AND NOT B) NOR (NOT C AND NOT D) IS
  WHEN '1' => F <= '1';
  WHEN '0' => F <= '0';
  WHEN OTHERS => NULL;
END CASE;

END PROCESS;

END behavioral;
```

Figure 8: VHDL Code for Small_ckt in Dataflow Style (Process and CASE Statement)

(g) Structural Design Style

- A structural architecture declaration is also called a hierarchical design approach
- The procedure to obtain a structural design is listed below:
 - Partition the design into manageable hardware blocks called COMPONENTS
 - Write complete VHDL code to define each COMPONENT with a dataflow design style or a behavioral design style
 - Write the library part and the entity declaration for the top level of the structural design
 - Declare all the internal signals and all the COMPONENTs in the declarative section of the ARCHITECTURE of the structural design
 - Instantiate the COMPONENTs (place and connect the COMPONENTs) in the statement section of the ARCHITECTURE of the structural design
- The internal details of the design entity Small_ckt has been decomposed as follows:
 - The components are enclosed by dotted lines
 - The components are given the arbitrary names (AND_1), (AND_2), and (OR_1)
 - The names of the components will be used as the names of the entities when writing the definitions for the components
 - The input and output signals of the components are called formal signals and are given the arbitrary signal names (i1), (i2), and (o1), and they will be used while writing the definitions for the components
 - The components are given the arbitrary component names (C1), (C2) and (C3), and they will be used when instantiating the components

Combinational Logic Design using VHDL

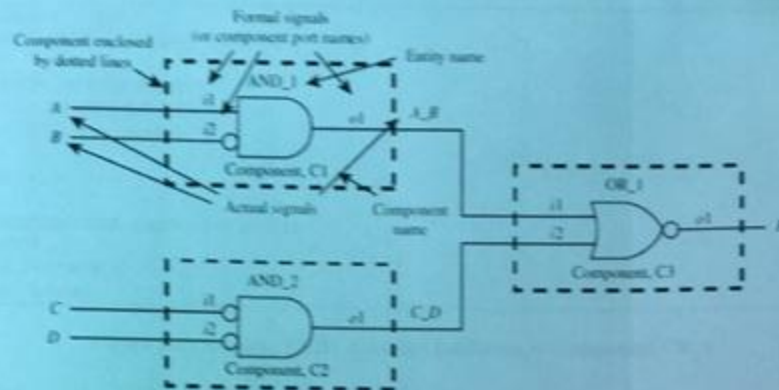


Figure 9: Small_ckt Partitioned into Components

- Components of Small_ckt need the following:
- Complete VHDL code for each component definition
 - The entity name for each component must be AND_1, AND_2, and OR_1
 - The formal signals (i1), (i2) and (o1) must be used to define the components

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY and_1 IS PORT (
    i1, i2: IN STD_LOGIC;
    o1: OUT STD_LOGIC
);
END and_1;

ARCHITECTURE dataflow OF and_1 IS
BEGIN
    o1 <= i1 AND NOT i2;
END dataflow;
```

Figure 10: Complete VHDL Code for Definition of Component AND_1

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY and_2 IS PORT (
    i1, i2: IN STD_LOGIC;
    o1: OUT STD_LOGIC
);
END and_2;

ARCHITECTURE dataflow OF and_2 IS
BEGIN
    o1 <= NOT i1 AND NOT i2;
END dataflow;
```

Figure 11: Complete VHDL Code for Definition of Component AND_2

Combinational Logic Design using VHDL

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY or_1 IS PORT (
    i1, i2: IN STD_LOGIC;
    o1: OUT STD_LOGIC
);
END or_1;

ARCHITECTURE dataflow OF or_1 IS
BEGIN
    o1 <= i1 NOR i2;
END dataflow;
```

Figure 12: Complete VHDL Code for Definition of Component OR_1

Note:

- ❖ The VHDL code for the component definitions must be placed in the same project as the VHDL code for the structural design i.e., design entity Small_ckt, to allow the VHDL code for the structural design to correctly compile
 - Save the component definitions in separate VHDL files having extensions (.vhd)
 - Create a new VHDL file with extension (.vhd) containing the structural design of Small_ckt
 - Create a new project in Xilinx ISE
 - Add the VHDL file containing the structural design of Small_ckt to the newly created project (using "Add Source" in Xilinx ISE)
 - Add the VHDL files containing the component definitions to the same project that contains the structural design of Small_ckt (using "Add Source" in Xilinx ISE)
 - Make the VHDL file containing the structural design of Small_ckt the top module (using "Set Module as Top" in Xilinx ISE)
- The ARCHITECTURE of Small_ckt contains the following items:
 - Two internal signals A_B and C_D have been defined, and they are visible only inside the ARCHITECTURE of Small_ckt
 - A MODE (IN, OUT, INOUT, BUFFER) is not required for internal signals
 - The COMPONENT declarations are placed in the declarative section of the ARCHITECTURE of Small_ckt
 - A COMPONENT declaration has exactly the same information as the ENTITY declaration for the COMPONENT except that it starts with the word COMPONENT and is terminated by END COMPONENT
 - Each COMPONENT is instantiated (placed and connected) within the statement section of the ARCHITECTURE of Small_ckt
 - The keyword PORT MAP is used to connect the formal signals (signals of components) with the actual signals (signals of the complete system)
 - A PORT MAP tells how a design entity is connected in the enclosing architecture
 - In a PORT MAP, the arrow symbol (=>) indicates the connection, and is read as "connected to"
 - The order of writing the instantiation statements is not important because they are concurrent statements
- The structural ARCHITECTURE declaration must be terminated by the keyword END
- COMPONENT instantiations cannot be placed inside a PROCESS

Combinational Logic Design using VHDL

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY Small_ckt IS PORT (
    A, B, C, D : IN STD_LOGIC;
    F : OUT STD_LOGIC
);
END Small_ckt;

ARCHITECTURE structural OF Small_ckt IS
    SIGNAL A_B, C_D: STD_LOGIC;
    COMPONENT and_1 IS PORT (
        i1, i2: IN STD_LOGIC;
        o1: OUT STD_LOGIC
    );
    END COMPONENT;

    COMPONENT and_2 IS PORT (
        i1, i2: IN STD_LOGIC;
        o1: OUT STD_LOGIC
    );
    END COMPONENT;

    COMPONENT or_1 IS PORT (
        i1, i2: IN STD_LOGIC;
        o1: OUT STD_LOGIC
    );
    END COMPONENT;

BEGIN
    C1: and_1 PORT MAP (i1 => A, i2 => B, o1 => A_B);
    C2: and_2 PORT MAP (i1 => C, i2 => D, o1 => C_D);
    C3: or_1 PORT MAP (i1 => A_B, i2 => C_D, o1 => F);
END structural;

```

Figure 13: Complete VHDL Code for Entity & Structural Architecture Declaration of Small_ckt

(h) VHDL Test Bench for Small_ckt

→ The input to Small_ckt has 4 inputs (A, B, C, D) and therefore results in 16 input possibilities:

Input Possibility	A	B	C	D
1	0	0	0	0
2	0	0	0	1
3	0	0	1	0
4	0	0	1	1
5	0	1	0	0
6	0	1	0	1
7	0	1	1	0
8	0	1	1	1
9	1	0	0	0
10	1	0	0	1
11	1	0	1	0
12	1	0	1	1
13	1	1	0	0
14	1	1	0	1
15	1	1	1	0
16	1	1	1	1

Table 1: Input Possibilities for Small_ckt

→ To avoid having to manually enter all 16 possibilities, looping technique has been used

Combinational Logic Design using VHDL

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY Small_ckt_tb IS
END Small_ckt_tb;

ARCHITECTURE behavioral OF Small_ckt_tb IS
    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT Small_ckt
    PORT(
        A : IN STD_LOGIC;
        B : IN STD_LOGIC;
        C : IN STD_LOGIC;
        D : IN STD_LOGIC;
        F : OUT STD_LOGIC
    );
    END COMPONENT;

    SIGNAL input_vector : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0000";
    SIGNAL output : STD_LOGIC;

BEGIN
    -- Instantiate the Unit Under Test (UUT)
    uut : Small_ckt PORT MAP(
        A => input_vector(3),
        B => input_vector(2),
        C => input_vector(1),
        D => input_vector(0),
        F => output
    );

    -- Stimulus process
    Stim_proc: PROCESS

    BEGIN

        FOR index IN 0 TO 15 LOOP
            input_vector <= std_logic_vector(to_unsigned(index,4));
            WAIT FOR 50 ns;
        END LOOP;

    END PROCESS;
END behavioral;
```

Figure 14: VHDL Test Bench for Small_ckt

- During each iteration of the loop, the simulator sets the input_vector equal to the iteration count
- Convert the index to unsigned, which is like STD_LOGIC_VECTOR except that the bits are interpreted as an unsigned number
 - Perform the conversion using the function TO_UNSIGNED declared in package IEEE.NUMERIC_STD
 - The first argument to the function TO_UNSIGNED is an integer
 - The second argument to the function TO_UNSIGNED specifies the bit width of the result
 - Convert the unsigned result to STD_LOGIC_VECTOR using a type conversion, which is similar to a type cast in a programming language like C or C++

Combinational Logic Design using VHDL

(i) Simulation Waveform for Small_ckt

- Figure 15 shows the output waveform for all possible input combinations
- The line "WAIT FOR 50 ns;" specifies that during simulation the PROCESS should wait for 50 nanoseconds
- After 16 iterations, the loop completes, and the PROCESS immediately restarts after the FOR loop exits
- To make the PROCESS stop after 16 iterations, an unconditional wait statement, i.e., "WAIT;" can be used

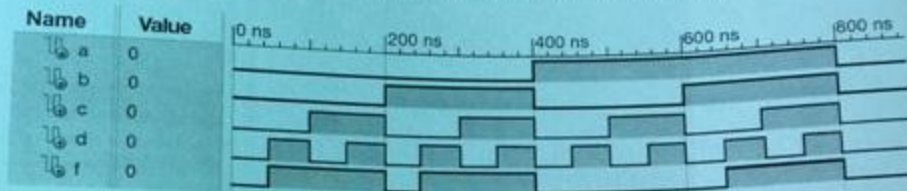


Figure 15: Simulation Waveform for Small_ckt